

SQL Server DBA Setup Guide

Last updated by | Haroun Ahmady | Apr 14, 2026 at 9:05 AM PDT

SQL Server DBA Setup Guide

Overview

This guide provides Database Administrators with complete instructions for setting up and maintaining the ClientIQ database on Microsoft SQL Server. It covers installation, security, backup strategies, and maintenance procedures.

Prerequisites

- SQL Server 2019 or later (2022 recommended)
- SQL Server Management Studio (SSMS)
- `sysadmin` server role membership
- Understanding of SQL Server security model

Step 1: Database Creation

Create Database

```

-- Create database with appropriate file placement
CREATE DATABASE ClientIQ
ON PRIMARY (
    NAME = 'ClientIQ_Data',
    FILENAME = 'D:\SQLData\ClientIQ.mdf',
    SIZE = 500MB,
    MAXSIZE = UNLIMITED,
    FILEGROWTH = 100MB
),
FILEGROUP [ClientIQ_Index] (
    NAME = 'ClientIQ_Index',
    FILENAME = 'D:\SQLData\ClientIQ_Index.ndf',
    SIZE = 200MB,
    MAXSIZE = UNLIMITED,
    FILEGROWTH = 50MB
)
LOG ON (
    NAME = 'ClientIQ_Log',
    FILENAME = 'E:\SQLLogs\ClientIQ.ldf',
    SIZE = 200MB,
    MAXSIZE = 2GB,
    FILEGROWTH = 50MB
);
GO

-- Set recovery model
ALTER DATABASE ClientIQ SET RECOVERY FULL;
GO

-- Enable query store for performance monitoring
ALTER DATABASE ClientIQ SET QUERY_STORE = ON;
ALTER DATABASE ClientIQ SET QUERY_STORE (
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    MAX_STORAGE_SIZE_MB = 1000
);
GO

USE ClientIQ;
GO

```

Database Settings

```

-- Set compatibility level
ALTER DATABASE ClientIQ SET COMPATIBILITY_LEVEL = 160; -- SQL Server 2022

-- Enable snapshot isolation for better concurrency
ALTER DATABASE ClientIQ SET ALLOW_SNAPSHOT_ISOLATION ON;
ALTER DATABASE ClientIQ SET READ_COMMITTED_SNAPSHOT ON;

-- Set auto-update statistics
ALTER DATABASE ClientIQ SET AUTO_UPDATE_STATISTICS ON;
ALTER DATABASE ClientIQ SET AUTO_CREATE_STATISTICS ON;

-- Page verification
ALTER DATABASE ClientIQ SET PAGE_VERIFY CHECKSUM;
GO

```

Step 2: Create Schema

Run Schema Script

The schema script is located at: database-scripts/v3/schema/sqlserver-schema-v3.sql

```
# Using sqlcmd
sqlcmd -S YourServer -d ClientIQ -i "sqlserver-schema-v3.sql" -o "schema_output.log"

# Verify execution
type schema_output.log
```

Verify Schema Creation

```
-- Check table count
SELECT COUNT(*) AS TableCount
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
-- Expected: 39

-- Check index count
SELECT COUNT(*) AS IndexCount
FROM sys.indexes
WHERE object_id IN (SELECT object_id FROM sys.tables)
  AND type > 0;
-- Expected: 90+

-- List all tables with row counts
SELECT
    s.name AS SchemaName,
    t.name AS TableName,
    p.rows AS RowCount
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id
JOIN sys.partitions p ON t.object_id = p.object_id AND p.index_id IN (0, 1)
ORDER BY t.name;
```

Step 3: Import Test Data (Optional)

For development/testing environments:

```
# Run test data script
sqlcmd -S YourServer -d ClientIQ -i "sqlserver-test-data-v3.sql" -o "data_output.log"
```

Verify import:

```
-- Check key tables have data
SELECT 'customer' AS TableName, COUNT(*) AS Records FROM customer
UNION ALL SELECT 'account', COUNT(*) FROM account
UNION ALL SELECT 'household', COUNT(*) FROM household
UNION ALL SELECT 'financial_transaction', COUNT(*) FROM financial_transaction
UNION ALL SELECT 'employee', COUNT(*) FROM employee;
```

Step 4: Security Configuration

Create Login and User

```

-- Create SQL Server login for the application
CREATE LOGIN svc_clientiq
WITH PASSWORD = 'StrongPassword123!',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON,
    DEFAULT_DATABASE = ClientIQ;
GO

USE ClientIQ;
GO

-- Create database user
CREATE USER svc_clientiq FOR LOGIN svc_clientiq;
GO

-- Create application role
CREATE ROLE app_clientiq;
GO

-- Add user to role
ALTER ROLE app_clientiq ADD MEMBER svc_clientiq;
GO

```

Grant Permissions

```

-- Grant schema-level permissions
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO app_clientiq;

-- Grant execute on stored procedures (if any)
GRANT EXECUTE ON SCHEMA::dbo TO app_clientiq;

-- Grant view definition for troubleshooting
GRANT VIEW DEFINITION ON SCHEMA::dbo TO app_clientiq;

-- Deny dangerous permissions
DENY ALTER ON SCHEMA::dbo TO app_clientiq;
DENY CREATE TABLE TO app_clientiq;
DENY CREATE PROCEDURE TO app_clientiq;
GO

```

Read-Only Access (for reporting)

```

-- Create read-only login for BI tools
CREATE LOGIN svc_clientiq_readonly
WITH PASSWORD = 'ReadOnlyPassword456!',
    CHECK_POLICY = ON,
    DEFAULT_DATABASE = ClientIQ;
GO

USE ClientIQ;
GO

CREATE USER svc_clientiq_readonly FOR LOGIN svc_clientiq_readonly;
GO

-- Grant read-only access
CREATE ROLE app_clientiq_readonly;
ALTER ROLE db_datareader ADD MEMBER app_clientiq_readonly;
ALTER ROLE app_clientiq_readonly ADD MEMBER svc_clientiq_readonly;
GO

```

Row-Level Security (Optional)

For region-based access control:

```
-- Create security predicate function
CREATE FUNCTION dbo.fn_securitypredicate(@region_id BIGINT)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_securitypredicate_result
WHERE @region_id IN (
    SELECT eb.branch_id
    FROM dbo.employee_branch eb
    JOIN dbo.branch b ON b.branch_id = eb.branch_id
    WHERE eb.employee_id = CAST(SESSION_CONTEXT(N'employee_id') AS BIGINT)
);
GO

-- Apply to customer table
CREATE SECURITY POLICY CustomerFilter
ADD FILTER PREDICATE dbo.fn_securitypredicate(branch_id)
ON dbo.customer
WITH (STATE = OFF); -- Enable when ready
GO
```

Step 5: Backup Strategy

Full Backup

```
-- Full backup script
BACKUP DATABASE ClientIQ
TO DISK = 'F:\SQLBackups\ClientIQ_Full.bak'
WITH FORMAT,
    INIT,
    NAME = 'ClientIQ Full Backup',
    COMPRESSION,
    STATS = 10,
    CHECKSUM;
GO

-- Verify backup
RESTORE VERIFYONLY FROM DISK = 'F:\SQLBackups\ClientIQ_Full.bak';
GO
```

Differential Backup

```
BACKUP DATABASE ClientIQ
TO DISK = 'F:\SQLBackups\ClientIQ_Diff.bak'
WITH DIFFERENTIAL,
    FORMAT,
    INIT,
    NAME = 'ClientIQ Differential Backup',
    COMPRESSION,
    STATS = 10;
GO
```

Transaction Log Backup

```

BACKUP LOG ClientIQ
TO DISK = 'F:\SQLBackups\ClientIQ_Log.trn'
WITH FORMAT,
    INIT,
    NAME = 'ClientIQ Log Backup',
    COMPRESSION,
    STATS = 10;
GO

```

Automated Backup Job

```

-- Create maintenance plan or use SQL Server Agent
-- Schedule: Full weekly (Sunday 2am), Diff daily (2am), Log every 15 min

```

```

USE msdb;
GO

```

```

-- Create backup job
EXEC sp_add_job @job_name = 'ClientIQ_Backup_Full';
EXEC sp_add_jobstep
    @job_name = 'ClientIQ_Backup_Full',
    @step_name = 'Full Backup',
    @subsystem = 'TSQL',
    @command = N'BACKUP DATABASE ClientIQ
        TO DISK = ''F:\SQLBackups\ClientIQ_Full_' +
            CONVERT(VARCHAR(8), GETDATE(), 112) + ''.bak''
        WITH COMPRESSION, CHECKSUM;',
    @database_name = 'master';

```

```

EXEC sp_add_schedule
    @schedule_name = 'Weekly_Sunday_2AM',
    @freq_type = 8,
    @freq_interval = 1,
    @freq_recurrence_factor = 1,
    @active_start_time = 020000;

```

```

EXEC sp_attach_schedule
    @job_name = 'ClientIQ_Backup_Full',
    @schedule_name = 'Weekly_Sunday_2AM';
GO

```

Step 6: Index Maintenance

Index Health Check

```

-- Check index fragmentation
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id) AS SchemaName,
    OBJECT_NAME(ips.object_id) AS TableName,
    i.name AS IndexName,
    ips.index_type_desc AS IndexType,
    ips.avg_fragmentation_in_percent AS Fragmentation,
    ips.page_count AS PageCount
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 10
    AND ips.page_count > 1000
ORDER BY ips.avg_fragmentation_in_percent DESC;

```

Index Rebuild Script

```
-- Rebuild fragmented indexes
DECLARE @TableName NVARCHAR(256);
DECLARE @IndexName NVARCHAR(256);
DECLARE @SQL NVARCHAR(MAX);

DECLARE index_cursor CURSOR FOR
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id) + '.' + OBJECT_NAME(ips.object_id),
    i.name
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 30
    AND ips.page_count > 1000
    AND i.name IS NOT NULL;

OPEN index_cursor;
FETCH NEXT FROM index_cursor INTO @TableName, @IndexName;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @SQL = 'ALTER INDEX [' + @IndexName + '] ON ' + @TableName + ' REBUILD WITH (ONLINE = ON);';
    PRINT @SQL;
    EXEC sp_executesql @SQL;
    FETCH NEXT FROM index_cursor INTO @TableName, @IndexName;
END

CLOSE index_cursor;
DEALLOCATE index_cursor;
```

Step 7: Statistics Maintenance

```
-- Update statistics for all tables
EXEC sp_updatestats;
GO

-- Update statistics with full scan for specific tables
UPDATE STATISTICS customer WITH FULLSCAN;
UPDATE STATISTICS account WITH FULLSCAN;
UPDATE STATISTICS financial_transaction WITH FULLSCAN;
GO
```

Step 8: Monitoring

Key DMVs for Monitoring

```

-- Expensive queries
SELECT TOP 20
    qs.total_elapsed_time / qs.execution_count AS avg_elapsed_ms,
    qs.execution_count,
    SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALength(st.text)
            ELSE qs.statement_end_offset
        END - qs.statement_start_offset)/2) + 1) AS statement_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE qs.total_elapsed_time > 1000000 -- > 1 second total
ORDER BY avg_elapsed_ms DESC;

-- Missing indexes
SELECT
    'CREATE INDEX IX_' + OBJECT_NAME(mid.object_id) + '_' +
        REPLACE(REPLACE(mid.equality_columns, '[', ''), ']', '') AS CreateStatement,
    migs.avg_user_impact,
    migs.user_seeks
FROM sys.dm_db_missing_index_groups mig
JOIN sys.dm_db_missing_index_group_stats migs ON mig.index_group_handle = migs.group_handle
JOIN sys.dm_db_missing_index_details mid ON mig.index_handle = mid.index_handle
WHERE mid.database_id = DB_ID()
ORDER BY migs.avg_user_impact DESC;

-- Blocking queries
SELECT
    r.blocking_session_id AS BlockingSession,
    r.session_id AS BlockedSession,
    r.wait_type,
    r.wait_time / 1000 AS wait_seconds,
    t.text AS BlockedQuery
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id > 0;

```

Create Monitoring Alert

```

USE msdb;
GO

-- Alert for long-running queries
EXEC sp_add_alert
    @name = 'Long Running Query',
    @message_id = 0,
    @severity = 0,
    @enabled = 1,
    @delay_between_responses = 300,
    @performance_condition = 'SQLServer:SQL Statistics|Batch Requests/sec|_Total|>|1000';
GO

```

Step 9: Connection Configuration

Connection String for Application


```
Server=sql-server.yourbank.local,1433;
Database=ClientIQ;
User Id=svc_clientiq;
Password=StrongPassword123!;
Encrypt=true;
TrustServerCertificate=false;
Connection Timeout=30;
```

Environment Variables

```
DATABASE_DIALECT=sqlserver
MSSQL_SERVER=sql-server.yourbank.local
MSSQL_DATABASE=ClientIQ
MSSQL_USER=svc_clientiq
MSSQL_PASSWORD=StrongPassword123!
MSSQL_PORT=1433
MSSQL_ENCRYPT=true
MSSQL_TRUST_SERVER_CERTIFICATE=false
```

Step 10: Disaster Recovery

Point-in-Time Recovery

```
-- Restore to specific time
USE master;
GO

-- Step 1: Restore full backup with NORECOVERY
RESTORE DATABASE ClientIQ_Restored
FROM DISK = 'F:\SQLBackups\ClientIQ_Full.bak'
WITH MOVE 'ClientIQ_Data' TO 'D:\SQLData\ClientIQ_Restored.mdf',
     MOVE 'ClientIQ_Log' TO 'E:\SQLLogs\ClientIQ_Restored.ldf',
     NORECOVERY,
     REPLACE;

-- Step 2: Restore differential with NORECOVERY
RESTORE DATABASE ClientIQ_Restored
FROM DISK = 'F:\SQLBackups\ClientIQ_Diff.bak'
WITH NORECOVERY;

-- Step 3: Restore log with STOPAT
RESTORE LOG ClientIQ_Restored
FROM DISK = 'F:\SQLBackups\ClientIQ_Log.trn'
WITH STOPAT = '2025-12-03 14:30:00',
     RECOVERY;
GO
```

Database Mirroring/Always On

For high availability, configure Always On Availability Groups:

```
-- Verify prerequisites
SELECT SERVERPROPERTY('IsHadrEnabled') AS AlwaysOnEnabled;

-- Backup for secondary
BACKUP DATABASE ClientIQ
TO DISK = 'F:\SQLBackups\ClientIQ_AlwaysOn.bak'
WITH FORMAT, INIT, COPY_ONLY;

BACKUP LOG ClientIQ
TO DISK = 'F:\SQLBackups\ClientIQ_AlwaysOn.trn'
WITH FORMAT, INIT;
```

Maintenance Checklist

Daily

- ☐ Verify backup completion
- ☐ Check SQL Agent job status
- ☐ Review error logs
- ☐ Monitor disk space

Weekly

- ☐ Run index maintenance
- ☐ Update statistics
- ☐ Review query performance
- ☐ Check for blocking issues

Monthly

- ☐ Test backup restoration
- ☐ Review security audit logs
- ☐ Analyze growth trends
- ☐ Update documentation

Quarterly

- ☐ Review and update maintenance plans
- ☐ Capacity planning review
- ☐ Security audit
- ☐ Performance baseline comparison

Troubleshooting

Common Issues

1. Connection Timeouts

- Check network connectivity
- Verify firewall rules (port 1433)
- Check SQL Server service status

2. Performance Degradation

- Review Query Store for regressions
- Check for index fragmentation
- Analyze wait statistics

3. Space Issues

- Shrink log file (if recovery model allows)
- Archive old data
- Check for transaction log growth

Support Contacts

Issue Type	Contact	SLA
Database down	DBA on-call	15 min
Performance issue	DBA team	1 hour
Security concern	Security team	30 min
Data corruption	DBA + Vendor	1 hour

Next Steps

1. [Environment Variables Reference](#)
2. [Troubleshooting Guide](#)